Abstractions, Dynamic Power, Amdahl's Law, and gem5 Setup

100 Points Possible

9/10/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

9/1/2026 to 9/15/2026

Details

Part A: Written Conceptual Problems (75 Points)

Problem 1: Latency vs. Throughput Dimensions [25 Points]

Background: This problem evaluates the core dimensions of processor performance—Response Time (Latency) and Throughput (Bandwidth)—as presented in *Lecture 3 (Slide 3)*. Answer each part with clear, direct systems-level reasoning:

- **Part A [8 Points]:** Suppose you replace a processor in a embedded controller with a new model that runs at exactly twice the clock frequency (e.g., scaling from 2.0 GHz to 4.0 GHz). Assuming the dynamic instruction count and execution behavior of a single-threaded control program remain completely unchanged, explain how this change affects: (1) the execution latency of that single program, and (2) the instruction throughput of the processor.
- **Part B [8 Points]:** Suppose you re-architect a server processor by adding 4 additional identical, independent CPU cores on the silicon die, while keeping the operating clock frequency of all cores unchanged. Explain how this architectural change affects: (1) the response time (latency) of a single-

threaded database lookup task, and (2) the overall transaction throughput of a highly parallel web-server workload.

- **Part C [9 Points]:** As a computer systems architect, describe how your target application workload dictates whether you optimize your design for latency or throughput. Provide one practical, real-world example of a system optimized primarily for response time, and one example of a system optimized primarily for bandwidth.

Problem 2: CMOS Dynamic Power Scaling & The Power Wall [25 Points]

Background: This problem explores the dynamic switching power equation (P_dynamic = C * V^2 * f * alpha) and the physical limitations of CMOS frequency scaling covered in *Lecture 3 (Slide 6 & Slides 9-10)*.

Scenario: A mobile application processor fabricated on a 14nm process node operates at a supply voltage of $V_0 = 1.20\ V$, active clock frequency $f_0 = 3.5\ GHz$, and consumes active dynamic switching power $P_0 = 25\ W$. Assume the dynamic activity factor (alpha) is constant throughout.

- **Part A [10 Points]:** The engineering team shrinks the core to a 7nm node, reducing active switching capacitance (C) by exactly 30%. If the supply voltage is successfully scaled down to $V_{new} = 0.95\ V$ and the operating frequency is increased to $f_{new} = 4.0\ GHz$, calculate the new active dynamic power consumption (P_new) of the core. Show your step-by-step ratio derivation.
- **Part B [10 Points]:** Suppose marketing demands a high-performance frequency boost to $f_{boost} = 4.8\ GHz$ on the same 7nm node. To support this frequency stably, engineers are forced to raise the supply voltage back to $V_{boost} = 1.15\ V$. Calculate the resulting dynamic power (P_boost) consumed by the core.
- **Part C [5 Points]:** Compare your results from Part A and Part B. Explain why the existence of a 'voltage floor' (~1.0V for sub-micron silicon) limits further processor frequency scaling. How does this demonstrate the physical origin of the Power Wall?

Problem 3: Speedup Limits and Amdahl's Law [25 Points]

Background: This problem exercises the mathematical bounds of Amdahl's Law and the sequential bottlenecks of heterogeneous computing discussed in *Lectures 4–5 (Slides 19-21)*.

Scenario: You are profiling a state-of-the-art machine learning workload. Profiling data shows that high-end matrix operations represent exactly **65%** of the application's overall execution time on a baseline general-purpose CPU (fraction enhanced, $f = 0.65$). To accelerate execution, your chip design team integrates a specialized Neural Processing Unit (NPU) that is measured to perform these matrix operations exactly **40 times faster** than the CPU (local speedup factor, $s = 40$).

- **Part A [10 Points]:** Calculate the overall system speedup achieved by integrating this NPU. Show your step-by-step application of Amdahl's Law and round your final answer to two decimal places.
- **Part B [8 Points]:** Suppose your software team works for months to optimize the NPU hardware, scaling its matrix execution speed infinitely (local speedup factor $s \rightarrow \infty$). Take the limit of Amdahl's Law as s approaches infinity and compute the theoretical maximum speedup ceiling of your overall system.
- **Part C [7 Points]:** Describe the physical or software bottleneck that prevents your overall speedup from exceeding this calculated ceiling. Based on this limitation, explain why computer architects prioritize 'making the common case fast' over accelerating highly complex, rarely executed sub-modules.

Part B: Hands-On gem5 Simulation Exercise (25 Points)

Objective: This lab checks your environment and compilation setup to ensure you can compile gem5, execute a pre-compiled binary in Syscall Emulation (SE) mode, and locate simulated hardware telemetry from *m5out/stats.txt*. This ensures both CS and EE students have a clean, operational workspace for upcoming assignments. Refer to *Syllabus Section 5* and the *gem5 Lab Guide* for setup references.

Step 1: SCons Simulator Compilation [5 Points]

Access your course-provided Linux environment. Navigate to the gem5 directory and compile the simulator for the RISC-V ISA. To leverage multi-threading and accelerate compile times, run SCons using the thread allocation flag (replace 4 with your host CPU core count):

```
scons build/RISCV/gem5.opt -j4
```

Your Task: Submit a screenshot of your terminal showing the successful completion of the scons build process, showing the final executable file compiled.

Step 2: Simple Executable Run in SE Mode [5 Points]

Run a pre-compiled RISC-V Hello World test binary under Syscall Emulation (SE) mode. Configure the run to utilize a *TimingSimpleCPU* (which stalls on cache accesses to provide basic timing realism) and enable classical cache configurations:

```
build/RISCV/gem5.opt configs/deprecated/example/se.py \
  --cpu-type=RiscvTimingSimpleCPU \
  --caches \
  -c tests/test-progs/hello/bin/riscv/linux/hello
```

Your Task: Submit a screenshot of your terminal output showing the simulation run output, concluding with 'Hello world!' printed by the simulated program.

Step 3: Navigating Telemetry & Metric Extraction [10 Points]

When a run finishes, gem5 writes all telemetry to *m5out/stats.txt*. Open this file in a text editor (such as vim, nano, or VS Code) to find, extract, and record the exact values for the 5 baseline hardware metrics listed in the table below.

gem5 Stat Key and Hardware Metrics

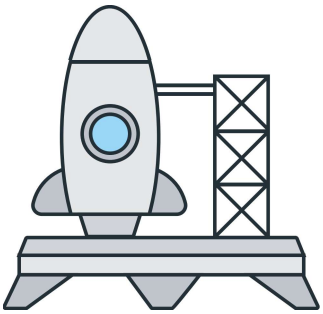
gem5 Stat Key (stats.txt)	Physical Hardware Metric Represented	Extracted Value (Hello World)
sim_ticks	Total simulated benchmark runtime in picoseconds (ps). (Include your conversion to nanoseconds (ns)).	*[Space for Student Entry]*
system.cpu.numCycles	Total simulated processor clock cycles spent processing the dynamic instruction stream.	*[Space for Student Entry]*
system.cpu.ipc	Instructions executed per clock cycle. (Compute its reciprocal to find the effective Cycles Per Instruction, CPI).	*[Space for Student Entry]*
system.cpu.dcache.overall_misses	Total cache lookup misses recorded on the L1 Data Cache interface during execution.	*[Space for Student Entry]*
system.cpu.dcache.overall_miss_rate	The fraction of L1 Data Cache references that resulted in misses, causing memory hierarchy stalls.	*[Space for Student Entry]*

Step 4: Microarchitectural Metric Reflection [5 Points]

Your Task: Provide a brief, 2-3 sentence reflection describing what these five metrics tell a processor designer about how the program executed on the simulated hardware. (For instance: what does the relationship between instructions, clock cycles, and cache misses show about processor execution efficiency?)

Choose a submission type

Upload | Google Drive | More



Choose a file to upload
File permitted: DOC, DOCX, PDF

or

Canvas Files